

Inteligência Artificial

Licenciatura em Engenharia Informática

Relatório do Trabalho Prático

Satisfação de Restrições



UNIVERSIDADE DE ÉVORA

Miguel Pombeiro, 57829 | Miguel Rocha, 58501

Departamento de Informática
Universidade de Évora
Abril 2026

Índice

1	Quadrado Latino	2
1.1	Problema Satisfação Restrições	2
1.2	Backtracking	4
1.3	Forward Checking	4
1.4	Modificação do Algoritmo	4
1.5	Outras dimensões	5
2	Futoshiki	6
2.1	Problema Satisfação Restrições	6
2.2	Backtracking	9
2.3	Forward Checking	9
2.4	Modificação Algoritmo	9
	Instruções para Correr o Problema	10

1 Quadrado Latino

1.1 Problema Satisfação Restrições

1.1.1 Estados

Os estados são representados por duas listas, sendo a primeira a lista das variáveis não instanciadas (VNI) e a segunda a das variáveis instanciadas (VI), $\text{Estado}(\text{VNI}, \text{VI})$.

1.1.2 Variáveis

As variáveis são representadas pelo predicado v , na seguinte forma:

$v(c(L, C), \text{Dominio}, \text{Valor})$

- O predicado $c(L, C)$ representa o conjunto de coordenadas no quadrado latino, sendo L a linha e C a coluna.
- O *Dominio* é uma lista de Prolog com dimensão igual ao tamanho do lado do quadrado latino, n :

$$\text{Dominio} = [1, 2, 3, \dots, n]$$

- O *Valor* representa o valor da variável, quando instanciada.

1.1.3 Restrições

Para este problema, as restrições são as seguintes:

- Em cada linha, todas as casas devem ter um número diferente.
- Em cada coluna, todas as casas devem ter um número diferente.

1.1.4 Estado Inicial

O estado inicial em Prolog, para o problema do Quadrado Latino para $n=3$ (3×3) é o seguinte:

```
estado_inicial(e([
    v(c(1,1), [1, 2, 3], _),
    v(c(1,2), [1, 2, 3], _),
    v(c(1,3), [1, 2, 3], _),
    v(c(2,1), [1, 2, 3], _),
    v(c(2,2), [1, 2, 3], _),
    v(c(2,3), [1, 2, 3], _),
    v(c(3,1), [1, 2, 3], _),
    v(c(3,2), [1, 2, 3], _),
    v(c(3,3), [1, 2, 3], _),
], [])).
```

Os estados iniciais para os tabuleiros com dimensões superiores podem ser encontrados no ficheiro "quadradolatino.pl".

1.1.5 Verifica Restrições

Para resolver este problema como um problema de satisfação de restrições, foi definido o seguinte predicado `ve_restricoes`, considerando as restrições aquelas definidas na Subsubseção 1.1.3:

```
ve_restricoes(e(_, Li)) :-
    ver_lista(Li).

ver_lista([]).

ver_lista([X | R]) :-
    \+ conflito(X, R),
    ver_lista(R).

conflito(_, []) :- fail.

conflito(v(c(L, C), _, V), [v(c(L1, C1), _, V)|_]) :-
    (L == L1; C == C1), !.

conflito(X, [_|R]) :-
    conflito(X, R).
```

1.1.6 Sucessor

O predicado `sucessor/2` utilizado foi o seguinte:

```
sucessor(e([v(P, D, _)|R], Li), e(R, [v(P, D, V)|Li])) :-  
    member(V, D).
```

1.2 Backtracking

Para o cálculo dos nós visitados, considerámos os seguintes pontos:

- Contámos o nó do estado inicial.
- Um nó é visitado após o algoritmo calcular os seus sucessores, mesmo que estes não verifiquem as restrições.
- O valor dos nós visitados para encontrar todas as soluções também incluiu os nós que foram visitados após terem sido encontradas todas as soluções (ou seja, os ramos que falharam mesmo após o algoritmo encontrar a última solução).

Sendo assim, para encontrar a primeira solução, foram visitados **22 nós** e para encontrar todas as soluções, foram visitados **247 nós**.

1.3 Forward Checking

Para o cálculo dos nós visitados, considerámos os pontos referidos na Subsecção 1.2.

Sendo assim, para encontrar a primeira solução, foram visitados **11 nós** e para encontrar todas as soluções, foram visitados **94 nós**.

1.4 Modificação do Algoritmo

Para a modificação do algoritmo de *forward checking*, foi utilizado o método *Minimum Remaining Values* (MRV), que escolhe as variáveis com menos valores possíveis no domínio. Para tal, a sua implementação passou a usar o predicado `sucessorMRV/2` em vez de `sucessor/2`.

```
sucessorMRV(e(Lni, Li), e(R, [v(P, D, V)|Li])) :-  
    escolhe_mrv(Lni, v(P, D, _), R),  
    member(V, D).
```

Para o cálculo dos nós visitados, considerámos os pontos referidos na Subsecção 1.2.

Sendo assim, para encontrar a primeira solução, foram visitados **10 nós** e para encontrar todas as soluções, foram visitados **88 nós**.

1.5 Outras dimensões

Para o cálculo dos nós visitados, considerámos os pontos referidos na Subsecção 1.2, bem como o algoritmo melhorado, ou seja, backtracking com MRV.

Tabela 1: Resolução dos quadrados latinos para $n = 4, 5, 6, 7, 8$ e 9 .

Tamanho	Nº Nós Primeira Solução	Nº Nós Todas as Soluções
$n = 4$	17	4745
$n = 5$	26	1712246
$n = 6$	37	Não terminou em tempo útil
$n = 7$	50	Não terminou em tempo útil
$n = 8$	65	Não terminou em tempo útil
$n = 9$	85	Não terminou em tempo útil

Não foi possível determinar o número de nós para todas as soluções dos quadrados latinos de $n=6$ a $n=9$. Isto deve-se ao rápido crescimento do número de soluções possíveis, como é possível verificar na Tabela 2. Sendo assim, o programa não conseguiu terminar em tempo útil para esses tamanhos de quadrados latinos.

Tabela 2: Número de soluções dos quadrados latinos para $n = 3, 4, 5, 6, 7, 8$ e 9 .

Tamanho	Número de Soluções
$n = 3$	12
$n = 4$	576
$n = 5$	161,280
$n = 6$	812,851,200
$n = 7$	61,479,419,904,000
$n = 8$	108,776,032,459,082,956,800
$n = 9$	5,524,751,496,156,892,842,531,225,600

Fonte: Wikipedia

2 Futoshiki

2.1 Problema Satisfação Restrições

2.1.1 Estados

Tal como na Subsubseção 1.1.1, os estados são representados por duas listas, sendo a primeira a lista das variáveis não instanciadas (VNI) e a segunda a das variáveis instanciadas (VI), Estado(VNI, VI).

2.1.2 Variáveis

Tal como na Subsubseção 1.1.2, as variáveis são representadas pelo predicado v , na seguinte forma:

$v(c(L, C), \text{Dominio}, \text{Valor})$

- O predicado $c(L, C)$ representa o conjunto de coordenadas no Futoshiki, sendo L a linha e C a coluna.
- O Dominio é uma lista de Prolog com dimensão igual ao tamanho do lado do Futoshiki, n :

$$\text{Dominio} = [1, 2, 3, \dots, n]$$

- O Valor representa o valor da variável, quando instanciada.

2.1.3 Restrições

Para este problema, as restrições são as seguintes:

- Em cada linha, todas as casas devem ter um número diferente.
- Em cada coluna, todas as casas devem ter um número diferente.
- As restrições de desigualdade entre duas casas adjacentes devem ser cumpridas (e.g. $4 > 3$, considerando a restrição $>$ entre a casa com o valor 4 e o valor 3).

A restrição das desigualdades, para o exemplo fornecido, foi lida de cima para baixo e da esquerda para a direita, representada na seguinte forma em Prolog:

```
d(c(1, 3), menor, c(2, 3)).  
d(c(1, 4), menor, c(2, 4)).  
d(c(2, 2), maior, c(2, 3)).  
d(c(2, 3), maior, c(2, 4)).  
d(c(3, 4), maior, c(4, 4)).
```

2.1.4 Estado Inicial

O estado inicial em Prolog, para o problema de construir um Futoshiki para $n=4$ (4x4) é o seguinte:

```
estado_inicial(e([  
    v(c(1,1), [1, 2, 3, 4], _),  
    v(c(1,2), [1, 2, 3, 4], _),  
    v(c(1,3), [1, 2, 3, 4], _),  
    v(c(1,4), [1, 2, 3, 4], _),  
  
    v(c(2,1), [1, 2, 3, 4], _),  
    v(c(2,2), [1, 2, 3, 4], _),  
    v(c(2,3), [1, 2, 3, 4], _),  
    v(c(2,4), [1, 2, 3, 4], _),  
  
    v(c(3,1), [1, 2, 3, 4], _),  
    v(c(3,2), [1, 2, 3, 4], _),  
    v(c(3,3), [1, 2, 3, 4], _),  
    v(c(3,4), [1, 2, 3, 4], _),  
  
    v(c(4,1), [1, 2, 3, 4], _),  
    v(c(4,2), [1, 2, 3, 4], _),  
    v(c(4,3), [1, 2, 3, 4], _),  
    v(c(4,4), [1, 2, 3, 4], _)  
], [])).
```

2.1.5 Verifica Restrições

Para resolver este problema como um problema de satisfação de restrições, foi definido o seguinte predicado `ve_restricoes`, considerando as restrições aquelas definidas na Subsubseção 2.1.3:

```
ve_restricoes(e(_, Li)) :-
    ver_desigualdades(Li),
    ver_lista(Li).

ver_desigualdades(Li) :-
    \+ (
        d(P1, maior, P2),
        member(v(P1, _, V1), Li),
        member(v(P2, _, V2), Li),
        V1 =< V2
    ),
    \+ (
        d(P1, menor, P2),
        member(v(P1, _, V1), Li),
        member(v(P2, _, V2), Li),
        V1 >= V2
    ).

ver_lista([]).
ver_lista([X | R]) :-
    \+ conflito(X, R),
    ver_lista(R).

conflito(_, []) :- fail.
conflito(v(c(L, C), _, V), [v(c(L1, C1), _, V)|_]) :-
    (L == L1; C == C1), !.

conflito(X, [_|R]) :-
    conflito(X, R).
```

2.1.6 Sucessor

Tal como na Subsubseção 1.1.6, o predicado `sucessor/2` utilizado foi o seguinte:

```
sucessor(e([v(P, D, _)|R], Li), e(R, [v(P, D, V)|Li])) :-
    member(V, D).
```

2.2 Backtracking

Para o cálculo dos nós visitados, considerámos os seguintes pontos:

- Contámos o nó do estado inicial.
- Um nó é visitado após o algoritmo calcular os seus sucessores, mesmo que estes não verifiquem as restrições.
- O valor dos nós visitados para encontrar todas as soluções também incluiu os nós que foram visitados após terem sido encontradas todas as soluções (ou seja, os ramos que falharam mesmo após o algoritmo encontrar a última solução).

Sendo assim, para encontrar a primeira solução, foram visitados **1345 nós** e para encontrar todas as soluções, foram visitados **1405 nós**. No entanto, é de notar que o problema tem apenas uma solução possível.

2.3 Forward Checking

Para o cálculo dos nós visitados, considerámos os pontos referidos na Subsecção 2.2.

Sendo assim, para encontrar a primeira solução, foram visitados **343 nós** e para encontrar todas as soluções, foram visitados **352 nós**. No entanto, é de notar que o problema tem apenas uma solução possível.

2.4 Modificação Algoritmo

Para a modificação do algoritmo de *forward checking*, foi utilizado o método *Minimum Remaining Values* (MRV), que escolhe as variáveis com menos valores possíveis no domínio. Para tal, a sua implementação passou a usar o predicado `sucessorMRV/2` em vez de `sucessor/2`.

```
sucessorMRV(e(Lni, Li), e(R, [v(P, D, V)|Li])) :-  
    escolhe_mrv(Lni, v(P, D, _), R),  
    member(V, D).
```

Para o cálculo dos nós visitados, considerámos os pontos referidos na Subsecção 2.2.

Sendo assim, para encontrar a primeira solução, foram visitados **91 nós** e para tentar encontrar todas as soluções, foram visitados **92 nós**. No entanto, é de notar que o problema tem apenas uma solução possível.

Instruções para Correr o Problema

Para correr ambos os problemas, `futoshiki.pl` e `quadradolatino.pl`, é necessário consultar o respetivo ficheiro, bem como o ficheiro do algoritmo a utilizar:

- `backtracking.pl`: Para o algoritmo de *backtracking* normal.
- `backFK`: Para o algoritmo de *backtracking* com *forward checking*.
- `backFKMelhorado.pl`: Para o algoritmo de *backtracking* com *forward checking* com MRV.

Posteriormente, é necessário chamar o predicado `p`.